

Fair Price Pipeline: Market → Executable → Strategy

Context

The engine currently computes a BlendedPrice in engine/market/price_aggregation.py using a simple VWAP weighted by 24h volume + two TWAP windows (5m, 1h). This is used by both arb detection and (soon) LP position centering. Two problems:

1. Wrong weights. Volume alone ignores spread width, active liquidity at the current tick, and staleness. Including Bybit P2P in VWAP math is unsound — it is peer-to-peer survey data, not an actionable quote. The fair price used for arb detection should not be moved by P2P listings.
2. Wrong abstraction. Arb detection needs a venue-neutral market signal (so it can tell when a venue deviates from fair value). LP positioning needs an inventory-aware center (so it tilts the range to reduce exposure). Using the same BlendedPrice object for both conflates concerns and risks feedback loops.

The FAIR_PRICE_V4_PLAN.md already defines the right three-tier concept (Market / Executable / Strategy). This plan implements it simply, without the latent Kalman filter (over-engineered for 2-3 live venues and a stable USDT/USDC basis) and with Bybit properly excluded.

Primary goal: build a composable, testable three-tier fair price pipeline that serves both arb detection and LP range centering from one coherent model.

Guiding constraints

- Simplicity first. No latent state-space model. No online estimation. Formulas must be auditable by inspection.
 - Each tier is independently testable and produces a named dataclass.
 - Arb detection reads only MarketFairPrice. It must never see inventory.
 - LP positioning reads StrategyFairPrice. Strategy fair price is never written to the DB or broadcast to the dashboard as a trading signal.
 - DEX imbalance uses signed swap flow (recent net buy vs. sell volume), not own-LP liquidity distribution. Own LP positions dominate nearby ticks in a thin pool — measuring them as a market signal is circular.
 - USDT/USDC basis: treated as fixed 1.0000 until evidence shows otherwise. No Kalman filter. Add a usdt_usdc_rate config knob so it can be corrected cheaply.
-

Tier 1 — MarketFairPrice

Purpose: The latent cNGN/USD price after filtering venue-specific noise. Venue-neutral.

Used by arb detection as the reference against which venue deviations are measured.

Weight formula

For each venue v :

$$w_v = \text{volume_weight}_v \times \text{liquidity_weight}_v \times \text{spread_quality}_v \times \text{recency}_v$$

volume_weight_v — 24h USD volume (already in RollingDexVolumeStore / Quidax depth).

This is the existing signal; it stays.

$\text{liquidity_weight}_v$ — proxy for how much depth is available at the current price:

- DEX: active_liquidity at current tick (from _POOL_CACHE in pool_state.py)
- CEX (Quidax): sum of bid + ask USD within ± 10 bps of mid (from walked orderbook)

spread_quality_v — $1 / \text{bid_ask_spread_bps}$. A spread of 5 bps gets weight $1/5$;

a spread of 50 bps gets $1/50$. For DEXes where bid $\equiv$ ask from sqrt price, use the implied fee tier as the minimum spread proxy (e.g. 15 bps for 0.15% pool).

recency_v — staleness decay: $\exp(-\lambda \times \text{age_seconds})$ where $\lambda \approx 0.008$

(half-life ≈ 87 s; reaches $\sim 1\%$ weight at 600s). Venues not updated in the last 10 minutes contribute near-zero weight.

Bybit — weight forced to zero. Kept in venue_prices display map but excluded from all fair-value math. Rename the constant in price_aggregation.py:

FAIR_VALUE_EXCLUDED = {"blockradar", "assetchain", "bybit"}.

Output type

```
@dataclass
class MarketFairPrice:
    price: Decimal          # cNGN/USD
    weights: dict[str, Decimal] # venue → normalized weight (for diagnostics/logs)
    confidence: float        # existing 0–0.9 formula
    timestamp: int
```

Tier 2 — ExecutableFairPrice

Purpose: Market fair price adjusted for short-horizon directional pressure via orderbook /

flow imbalance. This is a microprice-style offset: more buy-side pressure → price sits slightly above mid.

Imbalance signal (Answer A — weight, not filter)

Use imbalance as a weight that shifts the price, with a small coefficient so the maximum move

is ≤ 10 bps. The signal is informative; the risk is noise from a thin book, managed by the cap.

CEX imbalance (Quidax LOB) — Stoikov microprice:

Stoikov (2018) shows the microprice is a martingale estimator of next price — theoretically superior to mid-price. For the top-of-book level:

```
imbalance_cex = (bid_qty_best - ask_qty_best) / (bid_qty_best + ask_qty_best)
# ∈ [-1, 1]; positive = bid-heavy = upward pressure

microprice_cex = mid + (spread / 2) × imbalance_cex
```

Optionally extend to the full depth within 10 bps by using volume-weighted quantities:

```
bid_depth = Σ qty_i for bids within 10 bps of mid (quantity-only, not USD)
ask_depth = Σ qty_i for asks within 10 bps
imbalance_cex = (bid_depth - ask_depth) / (bid_depth + ask_depth)
```

DEX imbalance (signed swap flow, last N=20 swaps per pool):

```
imbalance_dex_v = (buy_vol_usd - sell_vol_usd) / total_vol_usd per pool
```

Swap history is already being emitted on-chain; store the last 20 signed flow observations per pool in a ring buffer (new field in pool state cache, no DB write needed).

Combined:

```
imbalance = w_cex × imbalance_cex + (1 - w_cex) × mean(imbalance_dex_v)
# w_cex ≈ 0.6 (CEX leads price discovery — see Literature section)

executable_price = market_price + (spread / 2) × imbalance
# Equivalent to: market_price × (1 + α × imbalance) with α = spread/2 / market_price
# Maximum shift bounded by spread/2 ≈ 5–15 bps in practice
```

Both w_{cex} and depth window should be config-driven so they can be tuned from backtester results.

Output type

```
@dataclass
class ExecutableFairPrice:
    price: Decimal
    market_price: Decimal # passthrough for audit
    imbalance_signal: Decimal # combined ∈ [-1, 1]
    imbalance_cex: Decimal | None
    imbalance_dex: dict[str, Decimal] # per pool
    timestamp: int
```

Tier 3 — StrategyFairPrice

Purpose: Executable price adjusted for the engine's own inventory position. Used only by

LP range centering. Never fed back into arb detection.

Inventory skew formula

Directly adapted from Avellaneda & Stoikov (2008), who show that a market maker's optimal reservation price is:

$$r = s - q \times \gamma \times \sigma^2$$

where s is market mid, q is signed inventory, γ is risk aversion, σ^2 is variance.

The (T-t) term from the original (finite-horizon session) is dropped since this is a continuous process.

Our simplified version:

```
net_cNGN = Σ_v cNGN_balance_v      # sum across all venues
target_cNGN = config.target_cNGN_inventory # e.g. 0 (flat) or some desired holding

normalized_imbalance = (net_cNGN - target_cNGN) / config.max_cNGN_inventory_scale

# Skew in bps: long cNGN → price lower (want to sell); short → price higher (want to buy)
raw_skew_bps = -β × normalized_imbalance      # β ≈ 10 bps per unit imbalance
skew_bps = max_skew × tanh(raw_skew_bps / max_skew) # sigmoid cap, max_skew = 20 bps

strategy_price = executable_price × (1 + skew_bps / 10_000)
```

The tanh cap replaces A-S's linear model: in a risk-bounded system we never want the skew to exceed max_skew even at extreme inventory, which the linear form doesn't guarantee.

Output type

```
@dataclass
class StrategyFairPrice:
    price: Decimal
    executable_price: Decimal      # passthrough
    skew_bps: Decimal             # signed, capped
    net_cNGN: Decimal
    timestamp: int
```

Files to create / modify

Create

- engine/market/fair_price.py — all three tiers:
 - MarketFairPriceCalculator
 - ExecutablePriceCalculator
 - StrategyPriceCalculator
 - Dataclasses: MarketFairPrice, ExecutableFairPrice, StrategyFairPrice

Modify

- engine/market/price_aggregation.py
 - Add bybit to FAIR_VALUE_EXCLUDED
 - Keep BlendedPriceCalculator for backwards compat during transition; it becomes a thin adapter over MarketFairPriceCalculator
 - Deprecate BlendedPrice.vwap in favour of MarketFairPrice.price
- engine/types.py
 - No new types needed at domain level — the fair price types are market-module-internal. Only add to types.py if arb or LP modules need to import them (avoid if possible).
- engine/arb/detection/cex_dex.py + dex_dex.py
 - Accept MarketFairPrice as an optional filter input. Use it to skip detection runs when the market price confidence is low (not to gate on inventory).
- engine/lp/ (range centering logic)
 - Accept StrategyFairPrice.price as the range center input. Currently uses BlendedPrice.vwap.
- engine/market/pool_state.py
 - Add a per-pool signed-flow ring buffer (last 20 swaps) for DEX imbalance signal.
- Config / env
 - Add FAIR_PRICE_ALPHA (imbalance coefficient, default 0.10)
 - Add FAIR_PRICE_MAX_SKEW_BPS (inventory skew cap, default 20)
 - Add FAIR_PRICE_BETA (inventory skew coefficient, default 10)
 - Add USDT_USDC_RATE (fixed basis correction, default 1.0000)

What to skip (vs. FAIR_PRICE_V4_PLAN.md)

Plan Phase	Decision
Phase 4: Kalman filter $F_t + B_t$	Skip. USDT/USDC basis is stable; estimate it as a config constant, not a latent state. Revisit only if basis spread exceeds 50 bps persistently.
Phase 4: latent state online update	
Bybit P2P in VWAP	Excluded. Reference display only.
DEX own-LP liquidity as imbalance signal	Excluded. Circular — own positions dominate nearby ticks. Use signed swap flow instead.
Phase 7 backtester adaptation	
Phase 7 backtester adaptation	Build in parallel with Tiers 1-2. Cannot validate imbalance coefficients without it.

Sequencing (priority order)

1. Tier 1 (MarketFairPrice) — highest immediate value; improves arb detection right away. Add spread quality + liquidity weight + staleness decay. Exclude Bybit. Wire into arb detection.
2. Tier 3 (StrategyFairPrice / inventory skew) — second priority; directly enables

inventory-aware LP centering. Formula is simple. Wire into LP range centering.

3. Tier 2 (ExecutableFairPrice / imbalance) — third priority; adds signal on top of a working Tier 1. Requires the signed-flow ring buffer in pool_state.

4. Backtester extension — run in parallel with Tier 2 to validate α and β coefficients. Cannot tune without it.

Literature Foundations

These papers ground each design decision. Read before implementation to understand the theoretical motivation and constraints.

Tier 1 (Market Fair Price)

Cont, Kukanov & Stoikov — "The Price Impact of Order Book Events" (2014)

[arxiv.org/abs/1011.6402]

Over short intervals, $\Delta P \approx \beta \times \text{OFI} / \text{depth}$. The linear OFI-price relationship holds across stocks and time scales. Grounds our claim that volume + depth weighting improves on VWAP.

Alexander (2025) — "Price Discovery and Efficiency in Uniswap Liquidity Pools"

[Wiley Journal of Futures Markets]

Shows that V3 efficiency is dramatically better than V2, with some pools rivaling Bitstamp.

But CEX still leads price discovery most of the time, especially for less-liquid pairs.

Implication: weight Quidax higher as the price-discovery anchor for cNGN.

Milionis et al. — "Automated Market Making and Loss-Versus-Rebalancing" (2022)

[arxiv.org/abs/2208.06046]

AMM sqrtPrice is only "correct" at the moment of an arb reset. Between resets, it drifts away from the efficient price, creating LVR for LPs. Implication: DEX sqrtPrice deserves lower weight in Tier 1 when no recent swap has occurred (staleness decay is the right model).

Tier 2 (Executable Fair Price / Imbalance)

Stoikov — "The Micro-Price: A High Frequency Estimator of Future Prices" (2018)

[SSRN 2970694]

The microprice $p^* = \text{mid} + (\text{spread}/2) \times q$ where $q = (\text{bid_qty} - \text{ask_qty})/(\text{bid_qty} + \text{ask_qty})$

is a martingale: it is the theoretically correct unbiased estimator of the next transaction price given current order book state. Mid-price is not a martingale (bid-ask bounce).

Directly drives the CEX imbalance formula in Tier 2.

Cont, Kukanov & Stoikov — OFI paper (same as above)

OFI at best bid/ask is the cleanest imbalance signal; deeper levels add noise. Start with top-of-book only and add depth levels only if backtester shows improvement.

Tier 3 (Strategy Fair Price / Inventory)

Avellaneda & Stoikov — "High-Frequency Trading in a Limit Order Book" (2008)

[people.orie.cornell.edu/sfs33/LimitOrderBook.pdf]

The reservation price $r = s - q \times \gamma \times \sigma^2$ is the market maker's internal "fair value"

adjusted for holding risk. Long inventory → lower reservation price (want to sell more).

Directly drives the inventory skew formula in Tier 3.

Note: the (T-t) term is dropped (continuous process assumption), and the tanh cap enforces a risk bound the original linear model lacks.

CEX-DEX Arbitrage Dynamics

Analysis of CEX-DEX Arbitrage Opportunities with Hidden Markov Models (ACM Web 2026)

[dl.acm.org/doi/10.1145/3774904.3792185]

HMM-identified regimes for CEX-DEX arb on calibrated Binance/Uniswap V3 data. Shows that price discovery shifts between CEX-leading and DEX-leading regimes. For thin pairs (like cNGN), CEX-leading is the dominant regime. Arb signal quality degrades during DEX-leading periods. Possible future extension: use a simple regime filter (time since last arb reset) to gate Tier 2 adjustments.

Milionis et al. + a16z LVR writeup

[a16zcrypto.com/posts/article/lvr-quantifying-the-cost]

The LP cost of AMM arb extraction. Useful for understanding why LP range centering with accurate fair price matters: a better strategy center reduces LVR by keeping the range tighter around the true price.

Verification

- Unit tests: Each calculator tested in isolation with synthetic venue data:
 - MarketFairPrice: verify stale venue gets near-zero weight; Bybit contributes zero
 - ExecutableFairPrice: bid-heavy book at $\alpha=0.10$ produces ≤ 10 bps upshift
 - StrategyFairPrice: long inventory produces negative skew capped at max_skew
- Integration smoke test: Instantiate all three tiers against live engine state; log all intermediate fields (weights, imbalance signals, skew_bps). Verify outputs are in valid cNGN/USD range (~0.0095–0.0110) and skew is small.
- Arb detection regression: After wiring MarketFairPrice, run the existing test suite (tests/test_arb_*.py) to confirm detection thresholds haven't shifted.
- Backtester comparison: Run walk-forward on historical V4 event data comparing Tier 1 fair price vs. current VWAP; measure mean absolute error vs. next-event realized price.